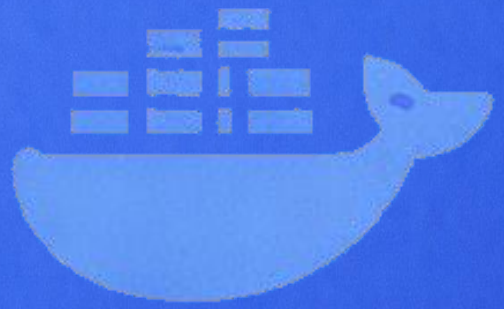
5%
NACIÓN
CLOUD

Curso de Docker

www.nacioncloud.com

Sección #1

Introducción



¿Qué es Docker?

- Docker es una plataforma que permite crear, ejecutar, actualizar y administrar contenedores.

- Un contenedor es un empaquetado que contiene el código y las dependencias de una aplicación en un formato estándar que permite su ejecución rápida y fiable en entornos de computo.

- El mismo contenedor siempre produce exactamente la misma aplicación y comportamiento de ejecución, sin importar dónde o quién la ejecute.

¿Para que usamos Docker?

Docker es usado con diferentes propósitos, por ejemplo:

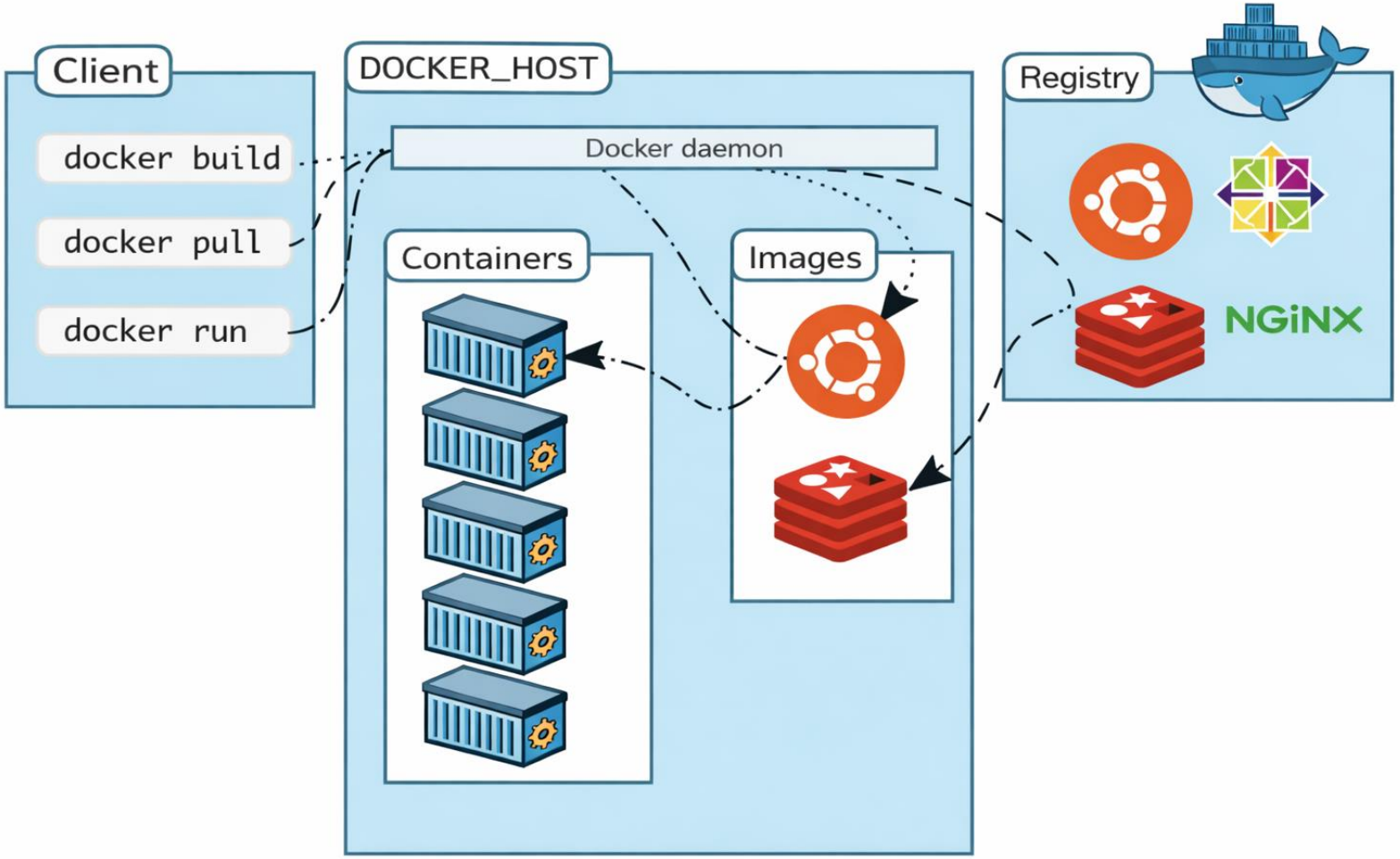
- Llevar código de aplicaciones web a ambientes de pruebas o productivos
- Desplegar trabajos en batch dentro de contenedores
- Empaquetar código con modelos de machine learning para enviar a entrenar
- Desplegar bases de datos de pruebas como postgres, redis, etc
- Contenedores acelerados con GPU para cargas de trabajo en computación grafica

LAB: Docker en Killer Coda

LAB: Instalación de Docker en Linux

LAB: Instalación de Docker Desktop en Windows

Arquitectura de Docker



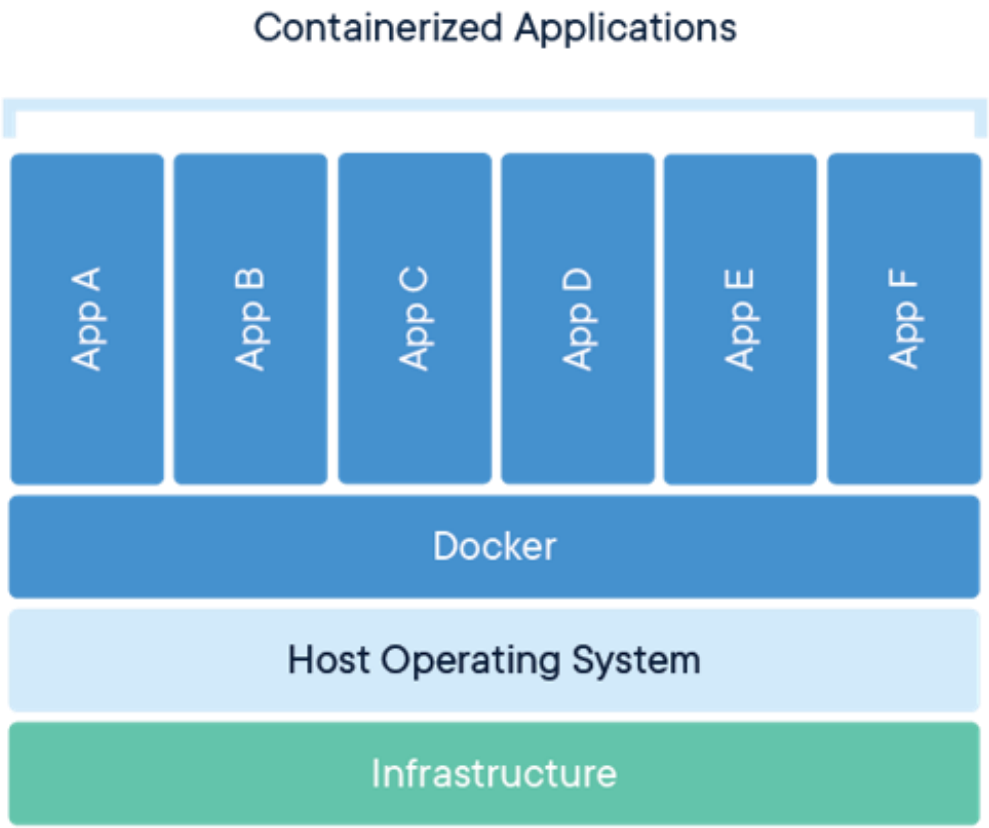
Arquitectura de Docker

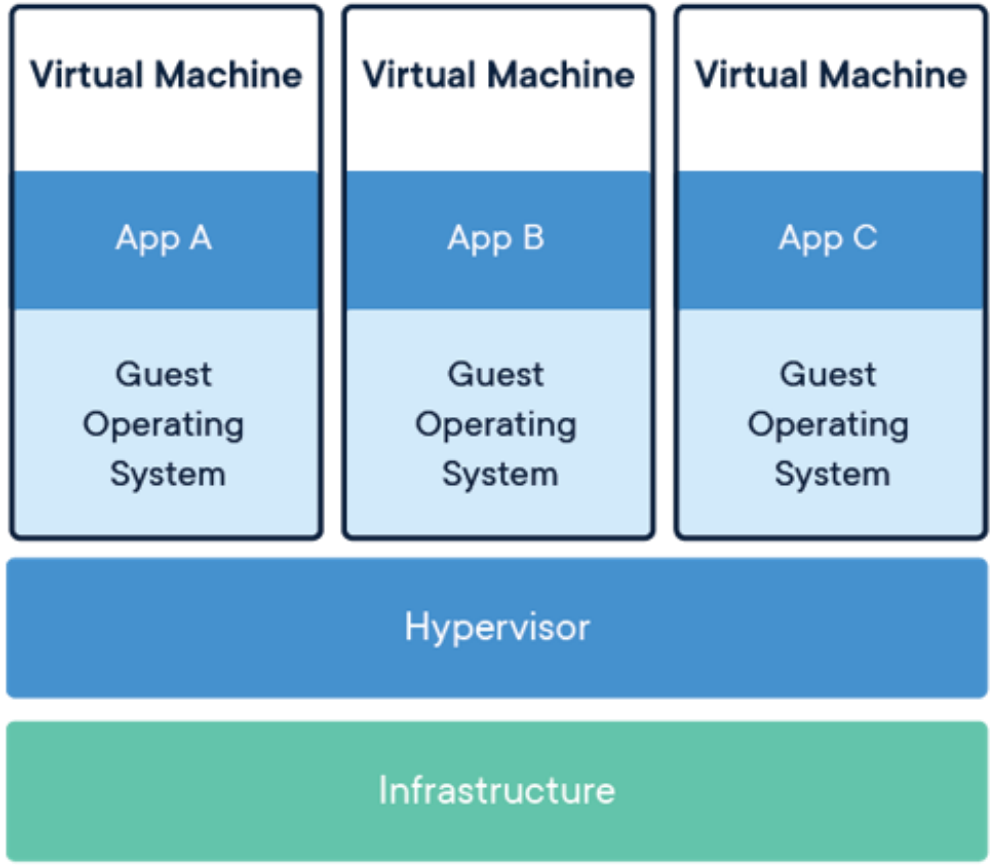
- **Docker Client:** Es la interfaz de línea de comandos (CLI) utilizada para interactuar con el motor de Docker (Docker Engine). Actúa como el intermediario entre el usuario y el demonio de Docker, enviando comandos a través de la API REST para gestionar contenedores e imágenes.

- **Docker Daemon (dockerd):** Es el proceso de servicio en segundo plano que actúa como el motor central de Docker, corriendo en el sistema operativo host. Se encarga de gestionar el ciclo de vida completo de los contenedores (creación, ejecución, monitorización), imágenes, redes y volúmenes, escuchando solicitudes API de la CLI.

- **Docker Registry:** Es un sistema de almacenamiento y distribución centralizado para imágenes de Docker. Actúa como una biblioteca o repositorio donde se suben (push) y descargan (pull) las imágenes necesarias para ejecutar contenedores, facilitando el intercambio de software, el control de versiones y la implementación en entornos de desarrollo o producción

Maquinas virtuales vs contenedores Docker





Maquinas virtuales vs contenedores Docker

Contenedor	Maquina Virtual
Bajo impacto en el sistema operativo, rápido, menor uso de espacio en disco.	Alto impacto en el sistema operativo, lento, mayor uso de espacio en disco.
Compartir, reconstruir y distribuir es sencillo.	Compartir, reconstruir y distribuir puede ser todo un reto.
Encapsula aplicaciones y ambientes en lugar de la maquina entera.	Encapsula la maquina entera en lugar de solo aplicaciones y ambientes.

Linux comandos básicos

- ls
- pwd
- cd
- touch
- mkdir
- cp
- rm
- echo
- cat
- ping
- curl

```
sara@pnap:~$ ls -n
total 64
drwxr-xr-x 2 1000 1000 4096 Mar  8  2024 Desktop
drwxr-xr-x 3 1000 1000 4096 Mar 22  2024 Documents
drwxr-xr-x 3 1000 1000 4096 May 10 11:11 Downloads
-rw-r--r-- 1    0    0   38 Jul 12 12:37 example1.txt
drwxr-xr-x 2 1000 1000 4096 Mar  8  2024 Music
drwxr-xr-x 2 1000 1000 4096 Mar  8  2024 Pictures
drwxr-xr-x 2 1000 1000 4096 Mar  8  2024 Public
drwxrwxr-x 3 1000 1000 4096 Aug 19 13:55 python
drwx------ 4 1000 1000 4096 Mar  8  2024 snap
drwxr-xr-x 2 1000 1000 4096 Mar  8  2024 Templates
-rw-rw-r-- 1 1000 1000   63 Jun 10 14:48 test1.txt
-rw-rw-r-- 1 1000 1000   22 May 31 13:28 test2.txt
-rw-rw-r-- 1 1000 1000   66 May 31 14:34 test3.txt
-rw-rw-r-- 1 1000 1000   43 May 31 14:45 test4.txt
-rw-rw-r-- 1 1000 1000   85 Jun  3 08:55 test5.txt
drwxr-xr-x 2 1000 1000 4096 Mar  8  2024 Videos
sara@pnap:~$
```

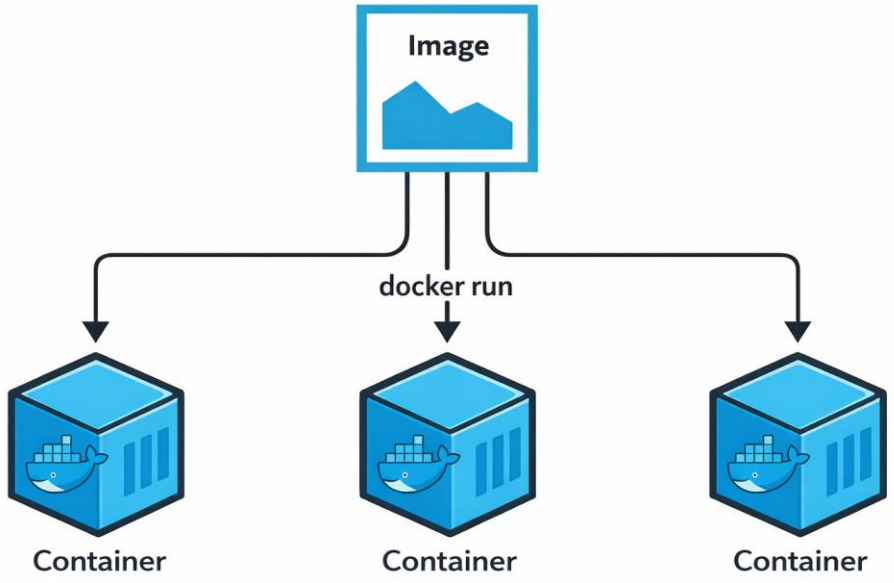
Sección #2

Imágenes y contenedores

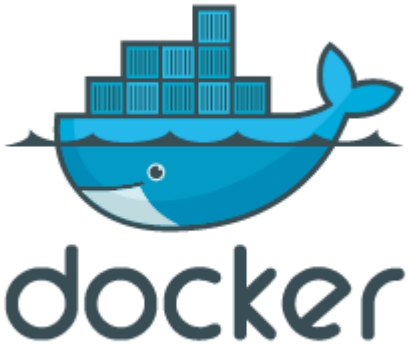
5%
NACIÓN
CLOUD

Imágenes y contenedores

- **Imágenes:** Son plantillas inmutables de solo lectura que contienen el código, bibliotecas y configuraciones necesarias para ejecutar una aplicación.
- **Contenedores:** Son las instancias de esas imágenes en ejecución, aisladas y ligeras, creadas a partir de estas imágenes. Es decir puedo usar una imagen para arrancar varios contenedores.

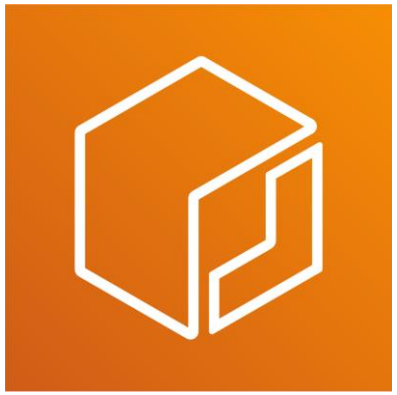


Repositorios de imágenes



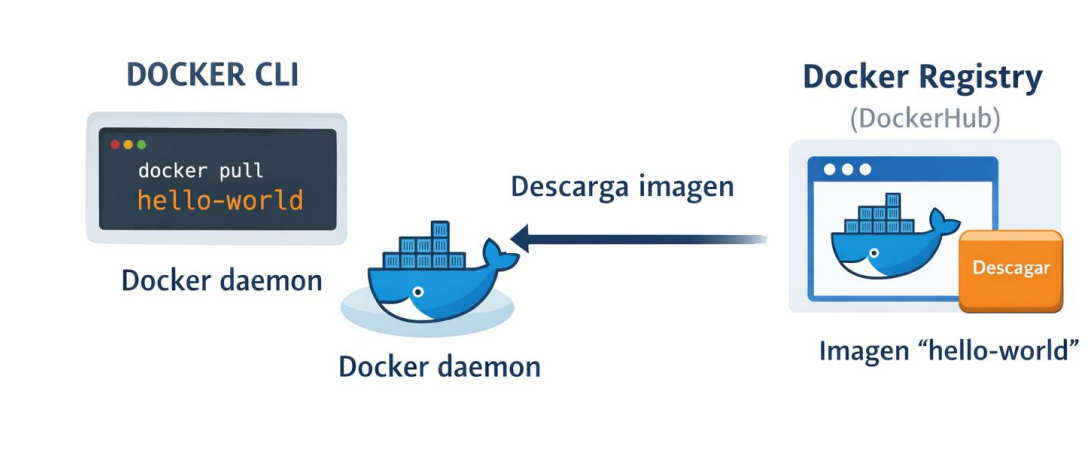
Docker Hub

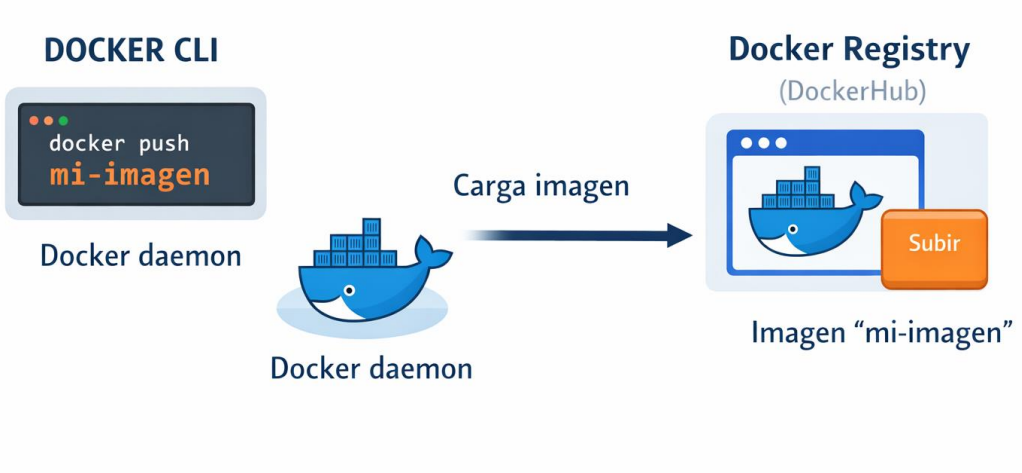
Quay



AWS ECR public gallery

Push y Pull





docker image push [OPTIONS] NAME[:TAG]

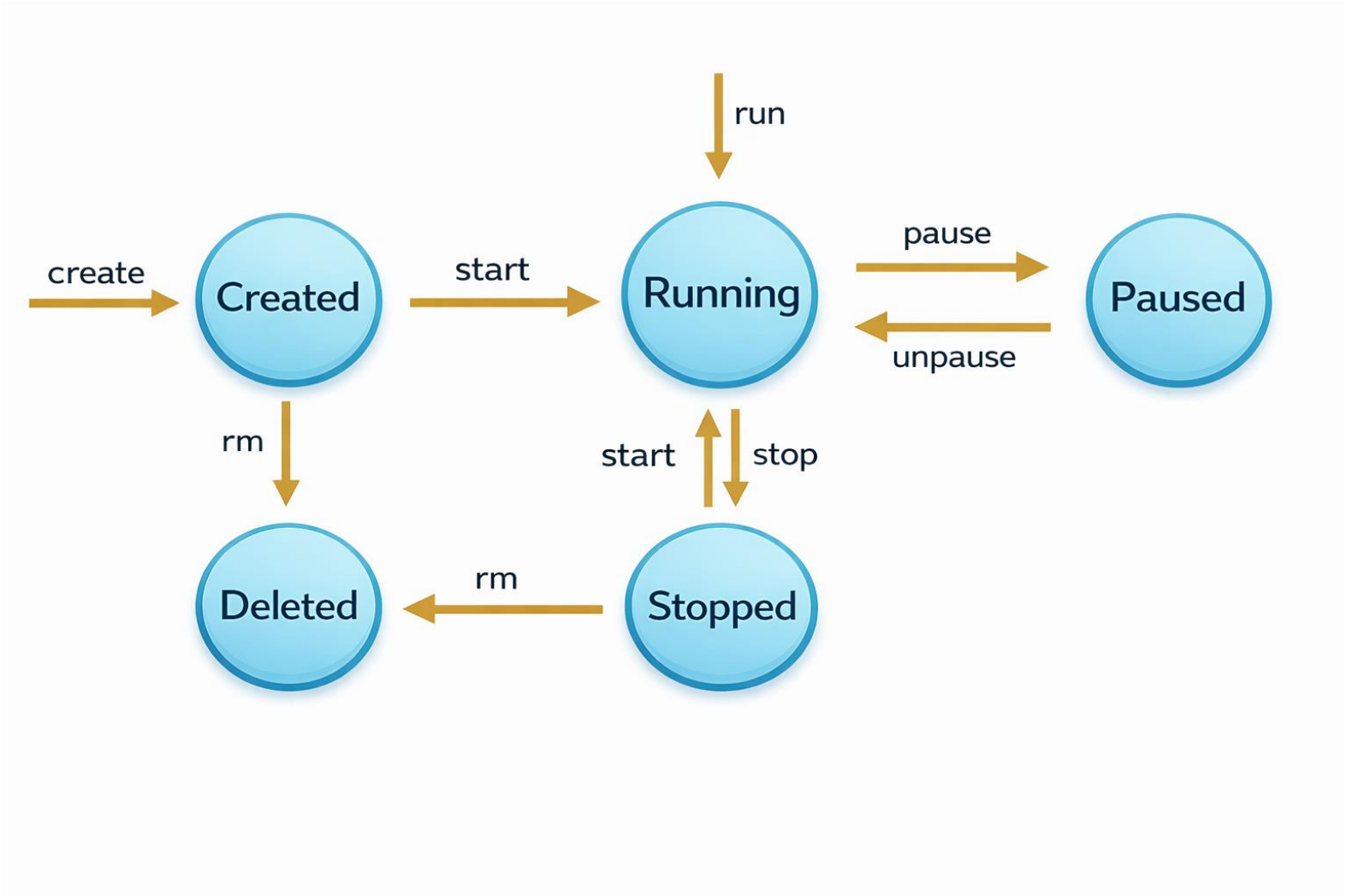
docker pull NAME[:TAG|@DIGEST]

Comandos en Docker

LAB: Trabajando con imágenes

LAB: Trabajando con contenedores

Ciclo de vida de un contenedor



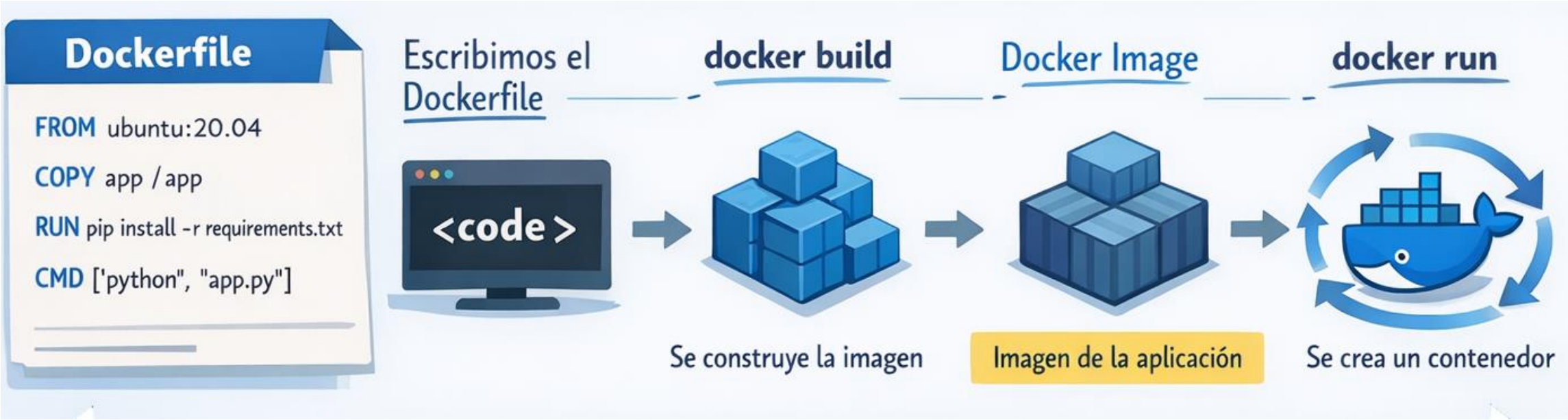
Sección #3

Dockerfile

5%
NACIÓN
CLOUD

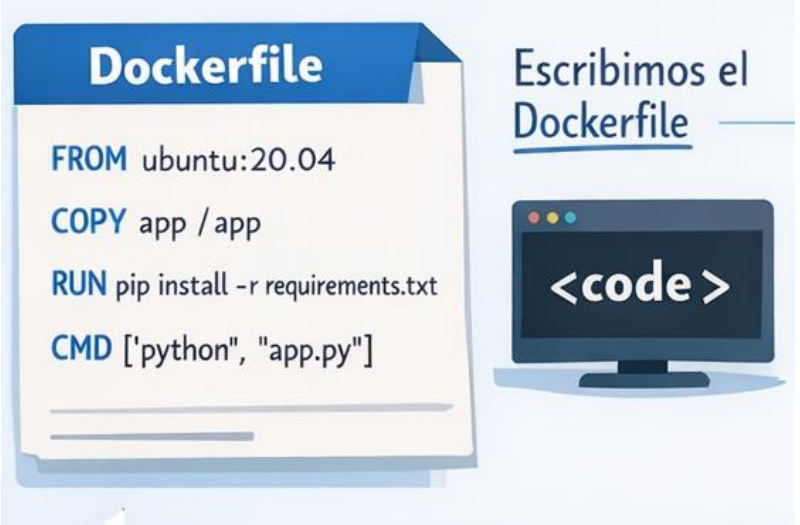
Dockerfile

- Un Dockerfile es un archivo o documento de texto simple que incluye una serie de instrucciones que se necesitan ejecutar de manera consecutiva para cumplir con los procesos necesarios para la creación de una nueva imagen.



Estructura

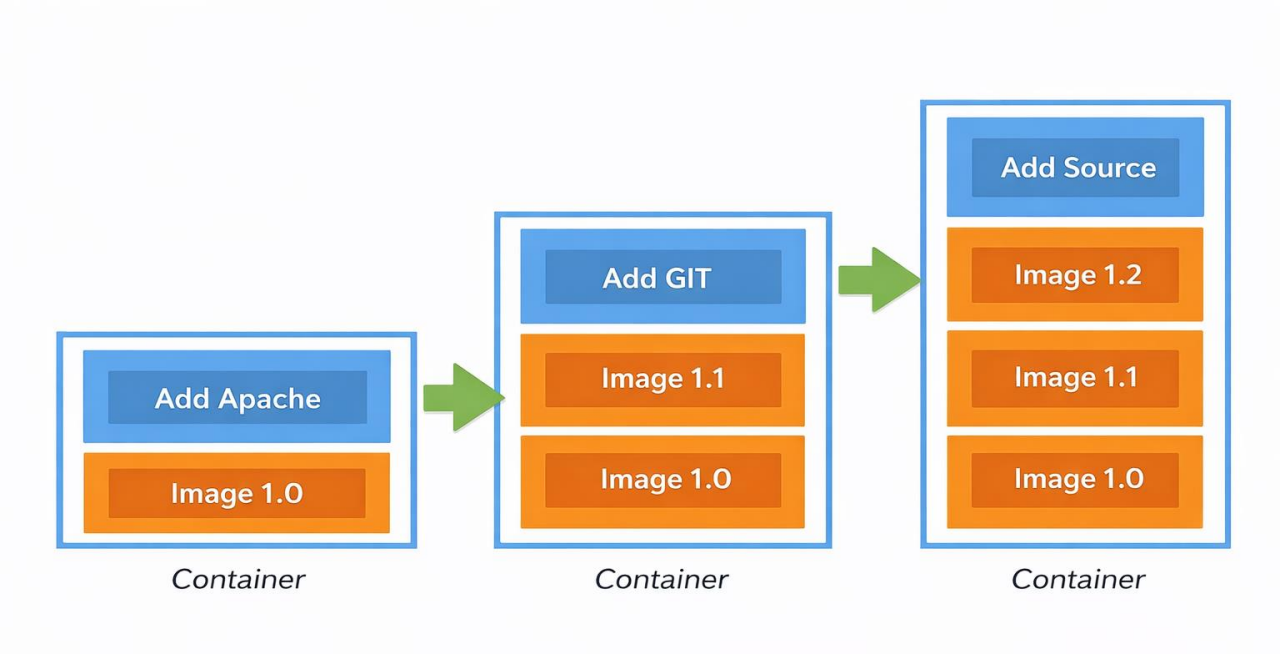
- FROM: Permite especificar la imagen base.
- MAINTAINER: Especifique el autor de una imagen.
- RUN: Ejecute comandos de compilación
- WORKDIR: Cambiar directorio de trabajo.
- COPY: Copiar archivos y directorios.
- ADD: Agregue archivos y directorios locales o remotos.
- EXPOSE: Describe en qué puertos escucha tu aplicación.
- VOLUME: Crea un volumen para montar.
- ENV: Establece variables de entorno.
- ARG: Utilice variables de tiempo de construcción.
- ENTRYPOINT: Especifica el ejecutable predeterminado.
- CMD: Especifique comandos predeterminados.



Capas

Hay dos principios importantes de las imágenes:

- Las imágenes son inmutables. Una vez que se crea una imagen, no se puede modificar. Solo se puede crear una nueva imagen o agregar cambios sobre ella.
- Las imágenes se componen de capas. Cada capa representa un conjunto de cambios en el sistema de archivos que agregan, eliminan o modifican archivos.



Valores en tiempo de compilación

Los argumentos de compilación de Docker (ARG) son variables que se pasan durante el proceso de compilación de la imagen para personalizarla sin modificar el contenido del Dockerfile.

Se especifican mediante el indicador --build-arg con el comando docker build y no están disponibles en el contenedor final en ejecución.

```
docker build --build-arg APP_VERSION=2.0 -t myapp:2.0 .
```

LAB: Trabajando con el Dockerfile

Diferencias variables de entorno (ENV) y los argumentos de construcción

Docker ARG (Build Argument)	Docker ENV (Environment Variable)
Sólo disponible durante el proceso de creación de la imagen.	Disponible durante el proceso de compilación Y en el contenedor en ejecución.
No se conserva en los metadatos/capas de la imagen final.	Persistió en la imagen final y fue accesible en tiempo de ejecución.
Útil para configuraciones dinámicas como números de versión o feature flags.	Útil para configuraciones de tiempo de ejecución como claves API, URL de bases de datos, etc.

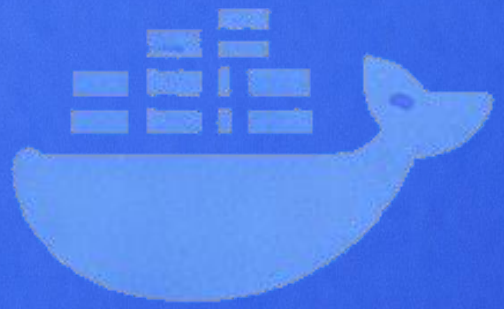
Sección #4

Optimización de imágenes

5%
NACIÓN
CLOUD

Sección #5

Almacenamiento



Almacenamiento

De manera predeterminada, todos los archivos creados dentro de un contenedor se almacenan en una capa de contenedor que permite la escritura, los datos escritos en la capa de contenedor no persisten cuando se destruye el contenedor.

La capa que permite la escritura es única por contenedor. No se pueden extraer fácilmente los datos de la capa que permite la escritura al host o a otro contenedor.

Docker admite los siguientes tipos de montajes de almacenamiento para almacenar datos fuera de la capa de escritura del contenedor:

- Volúmenes (gestionados por Docker)
- Montajes de enlace
- Montajes tmpfs

Volúmenes

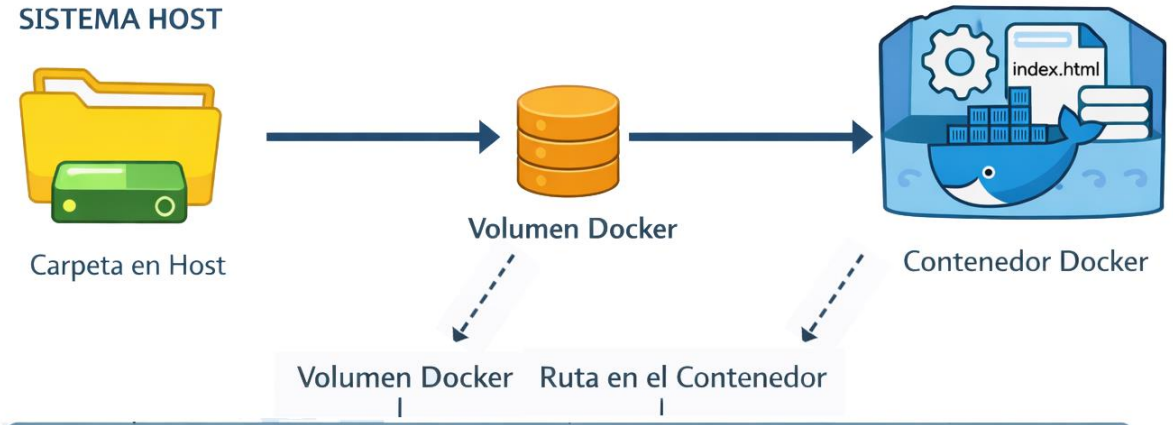
Los volúmenes son mecanismos de almacenamiento persistentes administrados por el demonio de Docker.

Conservan los datos incluso después de que se eliminen los contenedores que los utilizan.

Los datos del volumen se almacenan en el sistema de archivos del host, pero para interactuar con los datos del volumen, debe montar el volumen en un contenedor.

Los volúmenes son ideales para el procesamiento de datos críticos para las necesidades de almacenamiento a largo plazo.

Se pueden crear dos tipos de volúmenes: Nombrados y anónimos



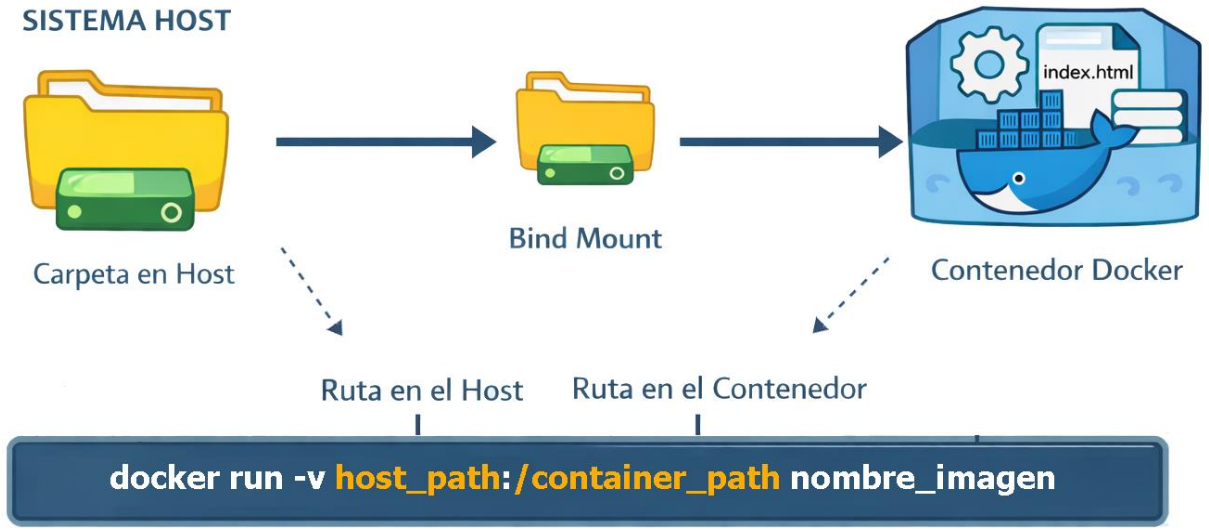

```
docker run -v demo_volume:/container_path nombre_imagen
```

LAB: Trabajando con volúmenes

Montajes de enlace

Los montajes de enlace crean un vínculo directo entre una ruta del sistema host y un contenedor, lo que permite el acceso a archivos o directorios almacenados en cualquier parte del host.

Utilice montajes de enlace cuando necesite poder acceder a archivos tanto desde el contenedor como desde el host.



LAB: Trabajando con montajes de enlace

Montajes tmpfs

Si está ejecutando Docker en Linux, puede usar los montajes tmpfs.

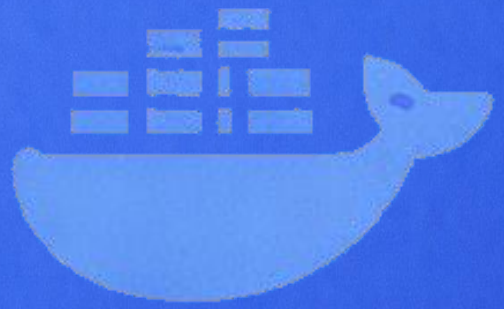
A diferencia de los volúmenes y los montajes de enlace, un montaje tmpfs es temporal y solo persiste en la memoria del host. Cuando el contenedor se detiene, el montaje tmpfs se elimina y los archivos escritos allí no se conservarán.

Los montajes tmpfs se utilizan mejor en los casos en los que no desea que los datos persistan ni en la máquina host ni dentro del contenedor. Esto puede deberse a razones de seguridad o para proteger el rendimiento del contenedor cuando su aplicación necesita escribir un gran volumen de datos de estado no persistentes.

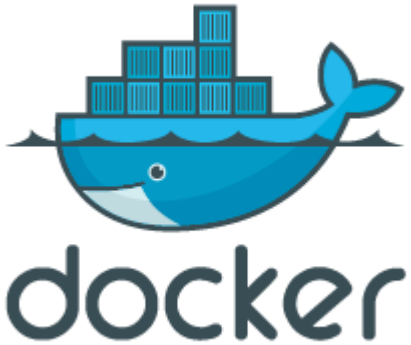
LAB: Copiando data

Sección #6

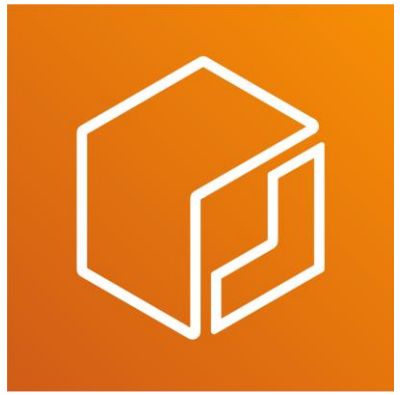
Registry



Registros para contenedores

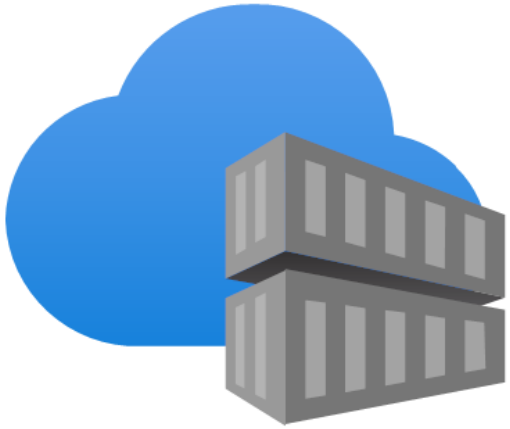


Docker Hub



AWS Elastic Container Registry

Google Container Registry



Azure Container Registry

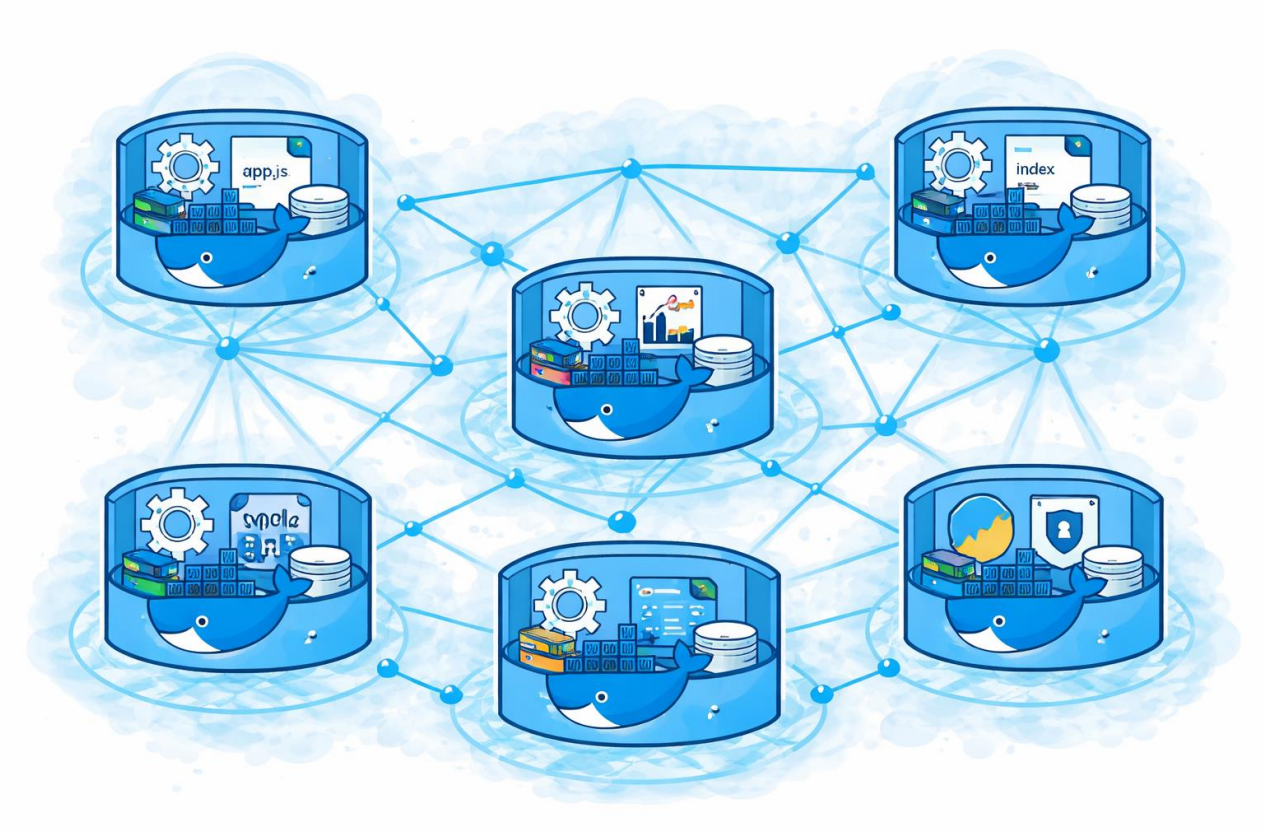
Sección #7

Redes

5%
NACIÓN
CLOUD

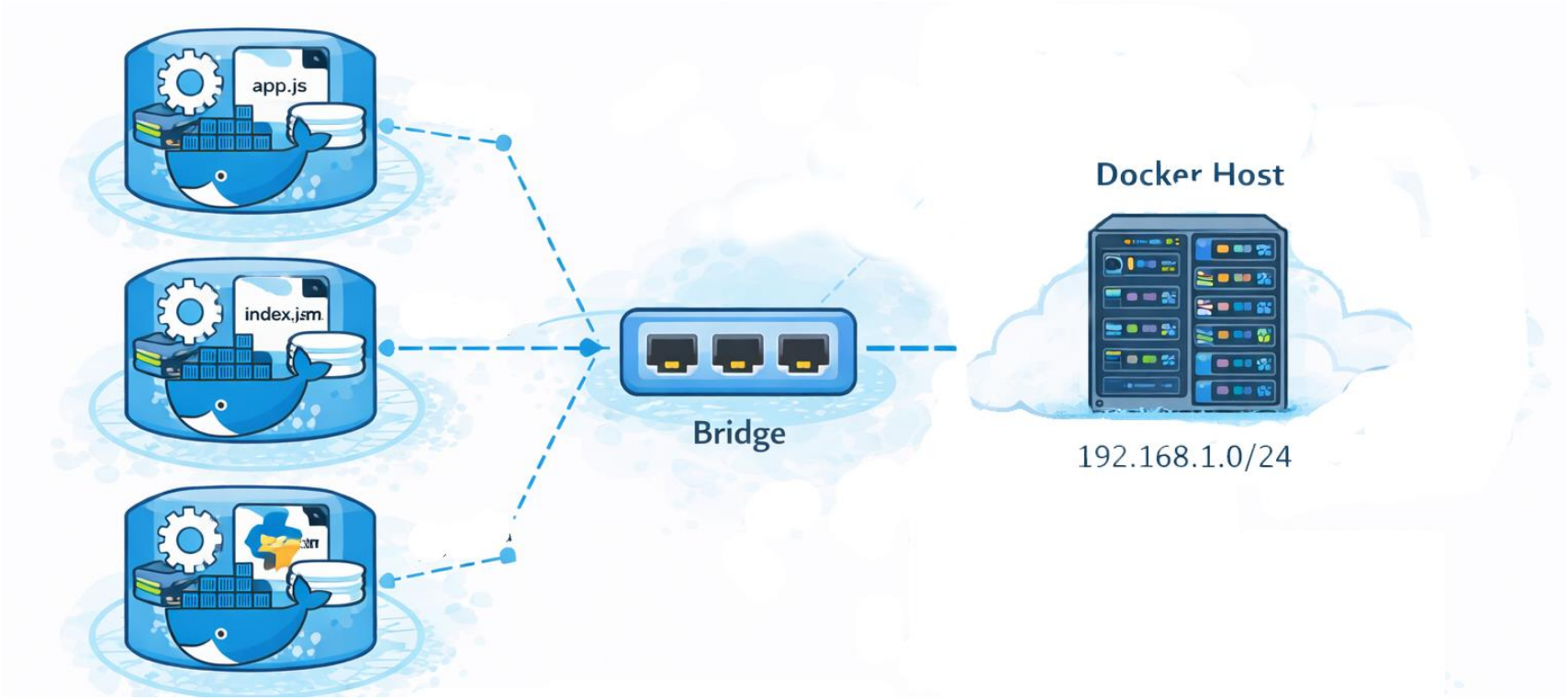
Redes

La red de contenedores se refiere a la capacidad de los contenedores de conectarse y comunicarse entre sí o con cargas de trabajo que no sean de Docker.



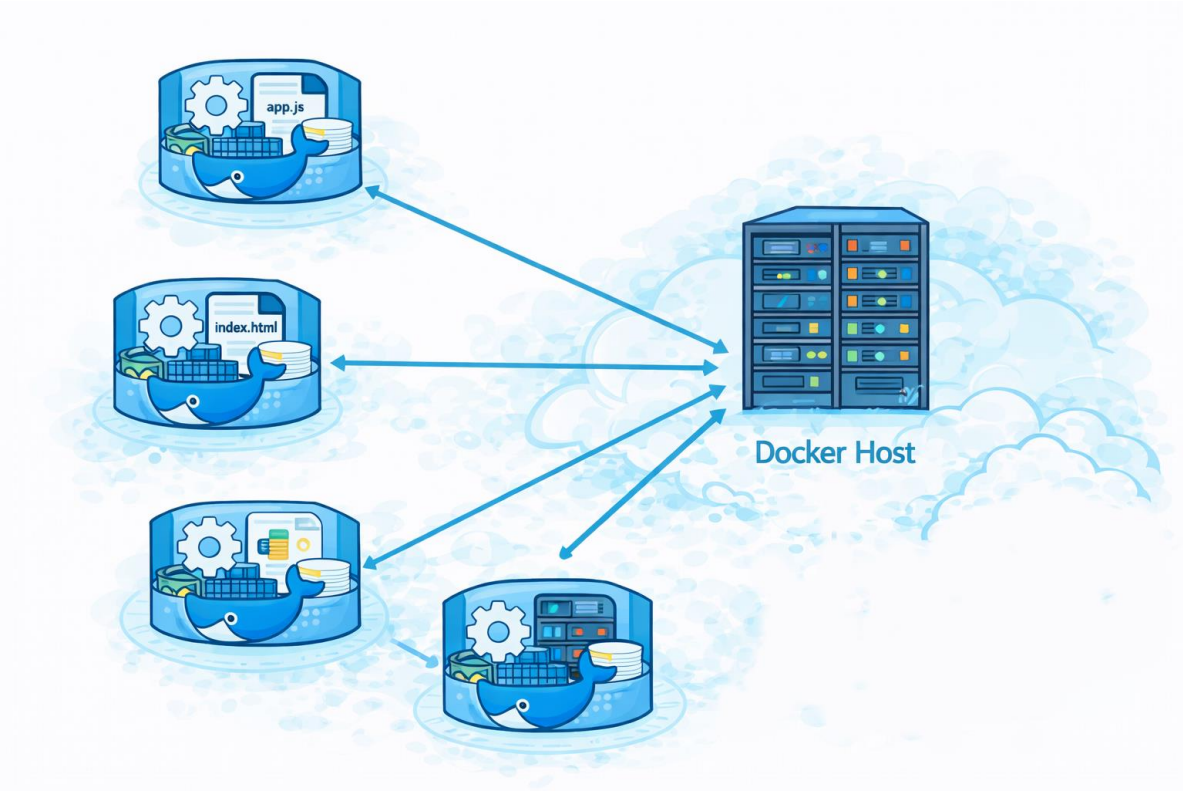
Redes bridge

Una red bridge es la red por defecto que Docker crea para que los contenedores puedan comunicarse entre sí dentro del mismo host.



Redes host

Una red host en Docker es un modo de red donde el contenedor usa directamente la red del sistema host, sin aislamiento.



Redes none

Con el driver none vamos a aislar un contenedor

Sección #8

Desplegando contenedores

5%
NACIÓN
CLOUD

5%
NACIÓN
CLOUD

Felicidades!

www.nacioncloud.com